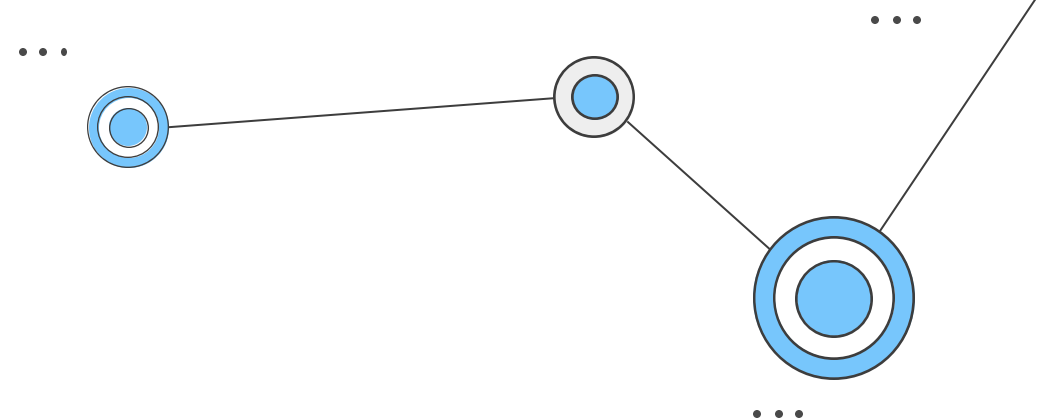
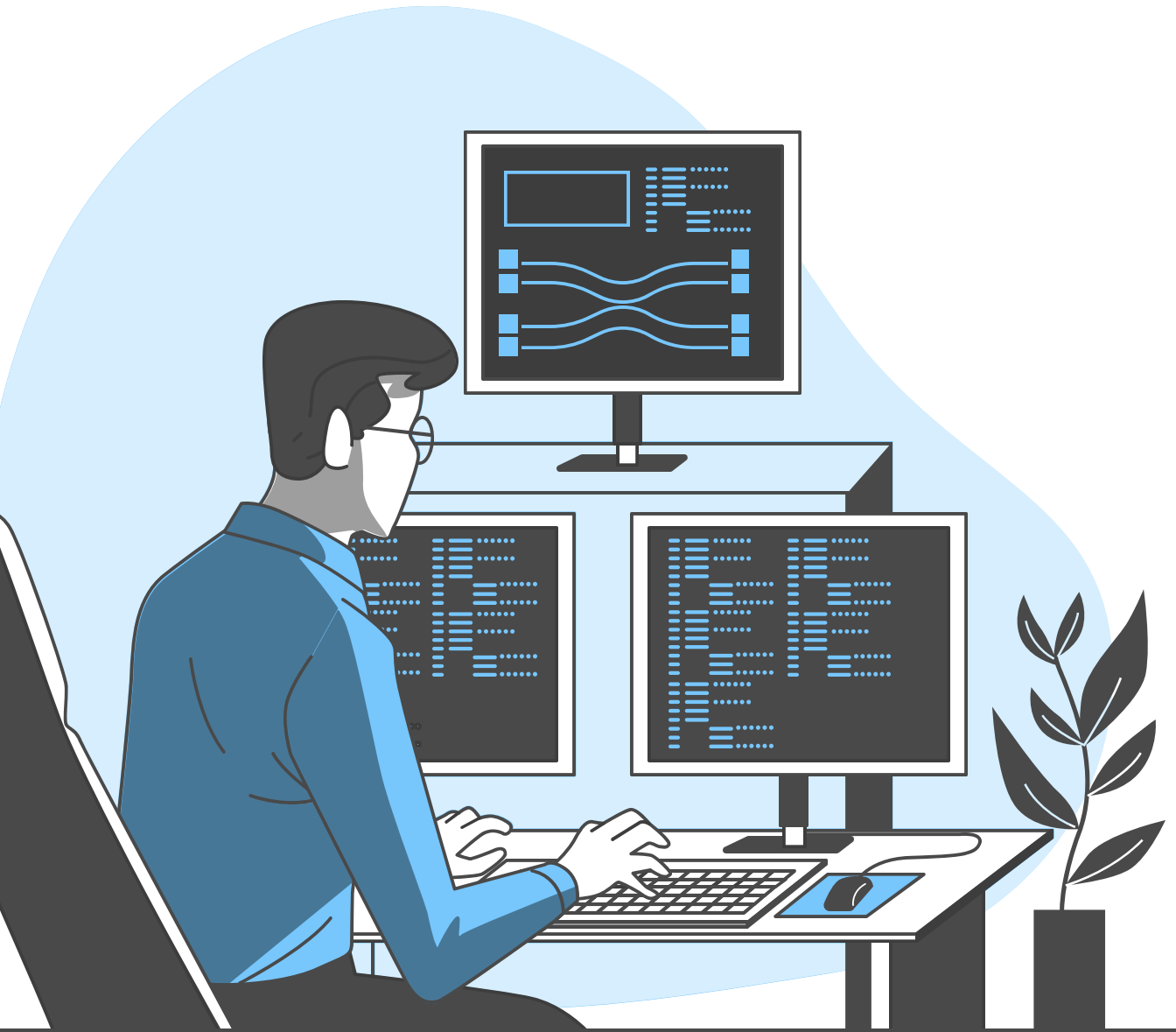


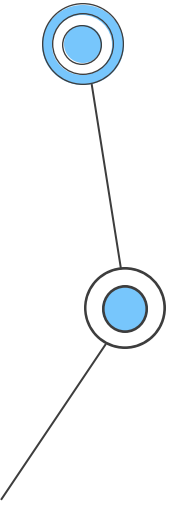
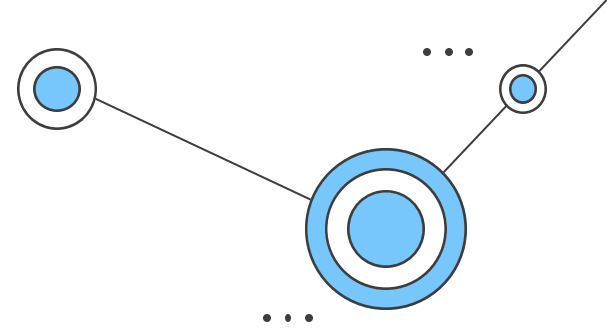


Conceitos Básicos da ES

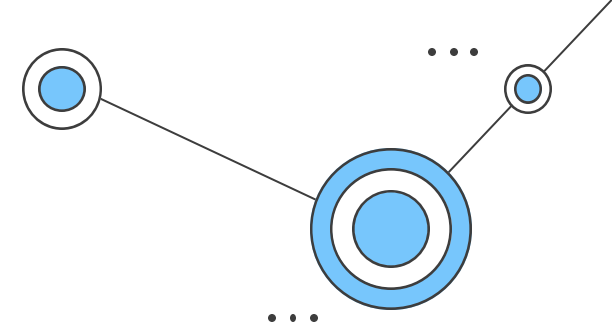


AGENDA

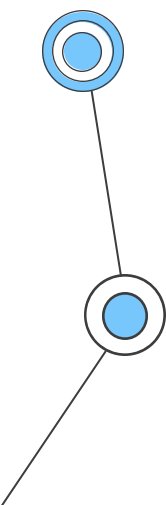
1. Conceitos Iniciais
2. Áreas de Conhecimento da ES



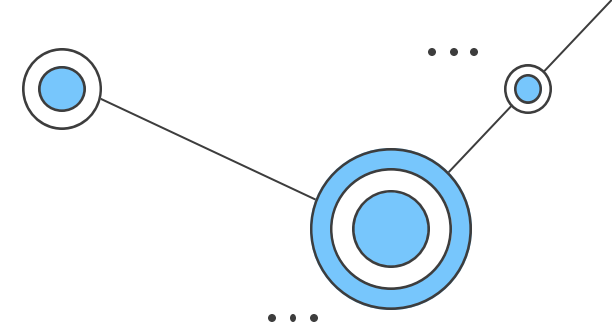
REFERÊNCIA



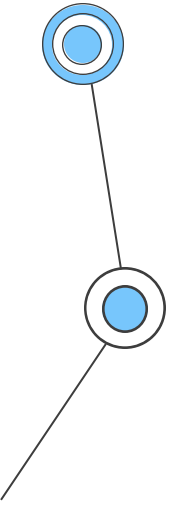
- SOMMERVILLE, Ian. Engenharia de software. 9. ed. São Paulo: Pearson Prentice Hall, 2011. xiii, 529 p. ISBN 9788579361081.
- VALENTE, Marco Tulio. Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade, Editora: Independente, 395 páginas, 2020.



CRISE DO SOFTWARE



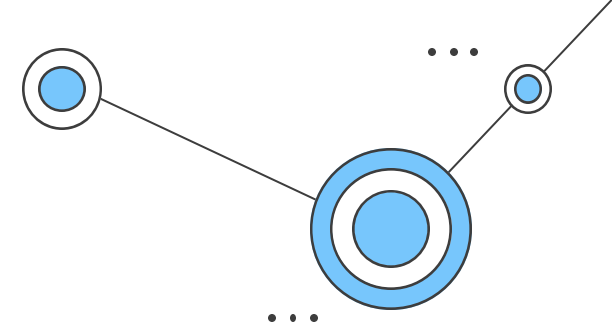
- Termo utilizado nos anos 1970, quando a ES era praticamente inexistente:
 - Projetos estourando orçamento;
 - Projetos estourando o prazo;
 - Software de baixa qualidade;
 - Software que não atende aos requisitos;
 - Projetos ingerenciáveis;
 - Código difícil de manter.



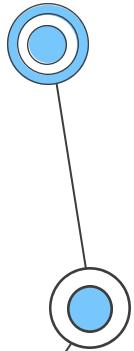
esse cenário MUDOU?

engenharia de software

ENGENHARIA DE SOFTWARE

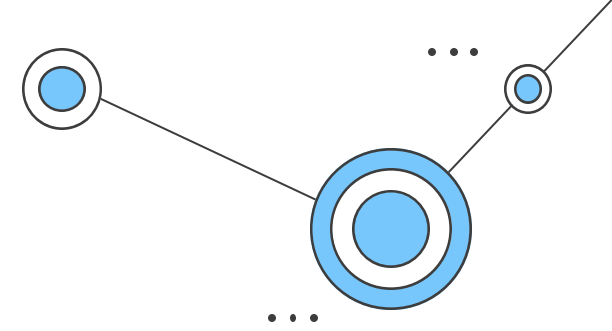


Engenharia de Software é uma **disciplina de engenharia** cujo foco está em **todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até sua manutenção, quando o sistema já está sendo usado.

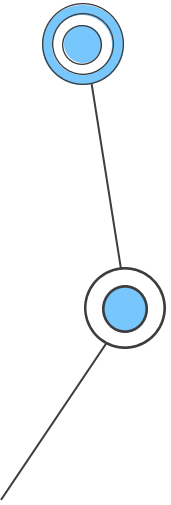


quais as diferenças da ES
para as outras áreas da
computação?

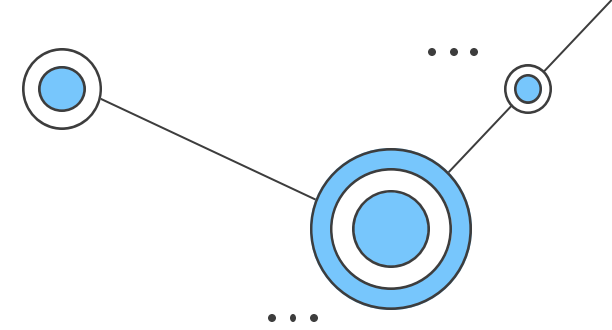
IMPORTÂNCIA DA ES



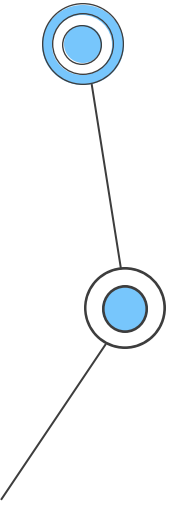
- Precisamos ser capazes de produzir **sistemas confiáveis com economia e rapidamente.**
- Geralmente, **é mais barato**, no longo prazo, **usar métodos e técnicas de engenharia de software** para os sistemas de software em vez de apenas escrever os programas como se fosse um projeto de programação pessoal.



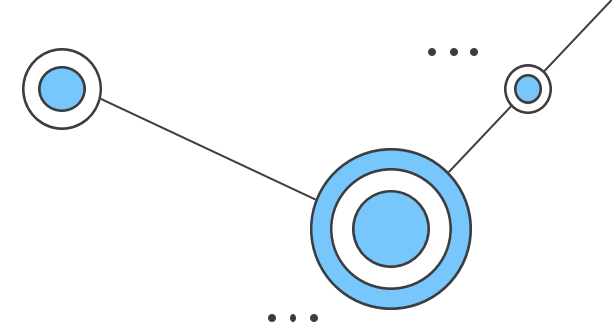
PRODUTOS DE SOFTWARE



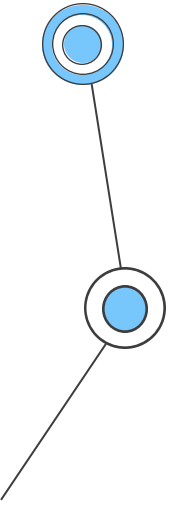
1. Produtos genéricos;
2. Produtos sob encomenda.



CUSTOS DE SOFTWARE

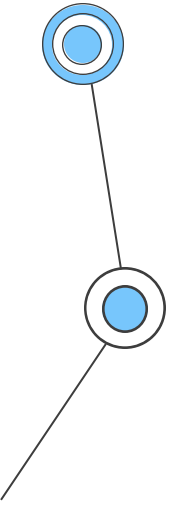
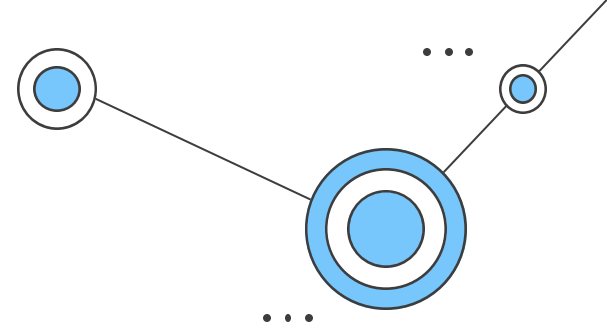


- A maior parte dos custos de software estão relacionadas aos **custos com mão de obra**;
- **Custa mais para se manter** um software do que para desenvolvê-lo.
- A engenharia de software está preocupada com que o desenvolvimento de software seja **custo efetivo**.

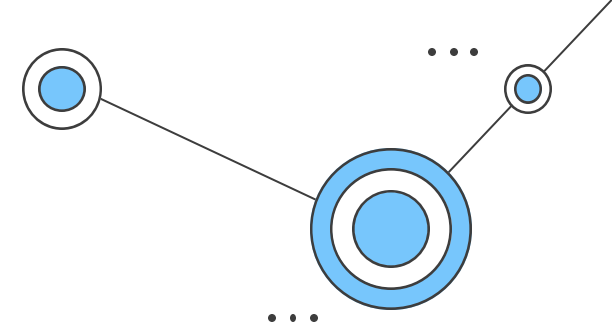


PRINCIPAIS DESAFIOS DA ES

- Volatilidade de requisitos;
- Diminuição do tempo de entrega;
- Desenvolvimento de software confiável.

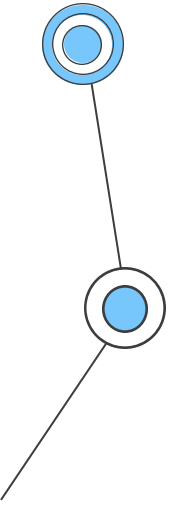


MELHORES TÉCNICAS E MÉTODOS



- Existem muitos tipos diferentes de sistemas de software e não existe um conjunto universal de técnicas de software aplicável a todos eles.

Depende...



MITOS
da engenharia de
software

**Se o cronograma atrasar,
podemos acrescentar
mais programadores e
ficarmos em dia?**

Se eu decidir por terceirizar o projeto de software, posso simplesmente relaxar e deixar a empresa realizá-lo.

**Uma definição geral dos
objetivos é suficiente para
começar a escrever os
programas.**

Os requisitos de software mudam continuamente, mas as mudanças podem ser facilmente assimiladas, pois o software é flexível.

**Uma vez feito o programa e
colocado em uso, nosso
trabalho está terminado.**

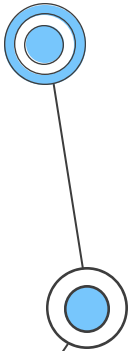
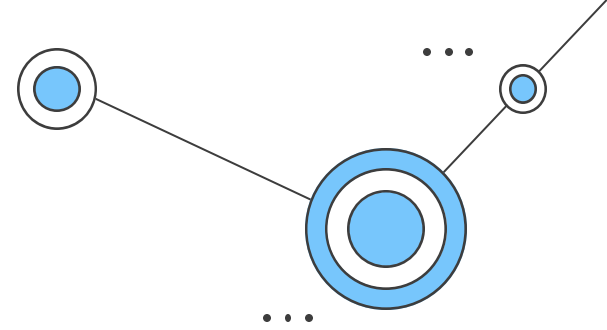
**Até que o programa entre em
funcionamento, não há
maneira de avaliar sua
qualidade.**

**O único produto passível de
entrega é o programa em
funcionamento.**

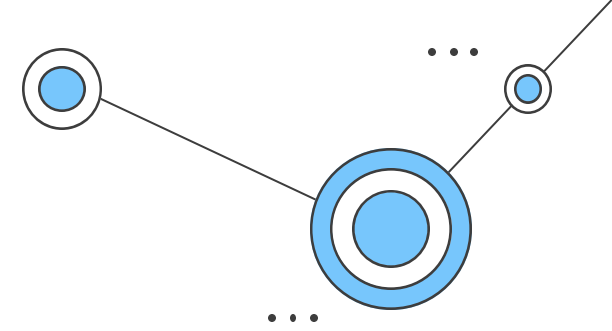
**A engenharia de software nos
fará criar documentação
volumosa e desnecessária e,
invariavelmente, irá nos retardar.**

ÁREAS DE CONHECIMENTO DA ES

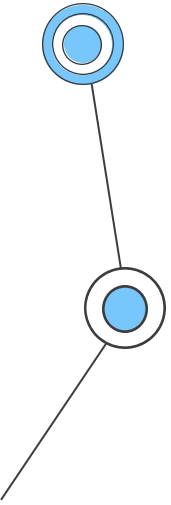
1. Processos e Modelos de Processos
2. Metodologias Ágeis
3. Requisitos de Software
4. Projeto de Software
5. Qualidade de Software
6. Teste de Software
7. Gerência de Configuração e Mudança
8. Manutenção de Software



1. PROCESSOS E MODELOS DE PROCESSOS



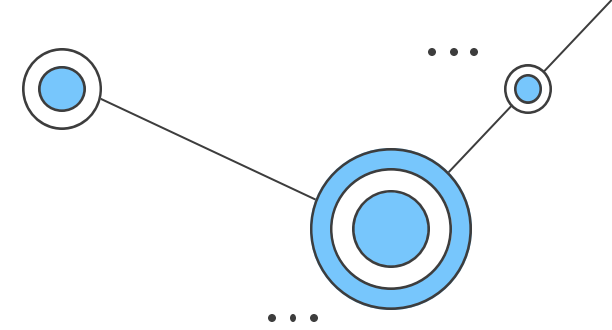
- Um **processo de software** é conjunto de atividades relacionadas que leva a produção de um produto de software.
- Existem **muitos processos de software diferentes**, mas todos devem incluir **quatro atividades fundamentais** para a engenharia de software.



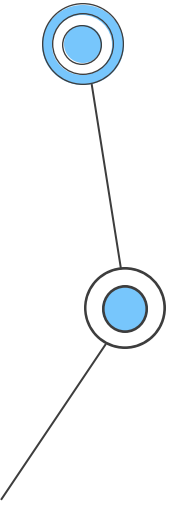
1. PROCESSOS E MODELOS DE PROCESSOS



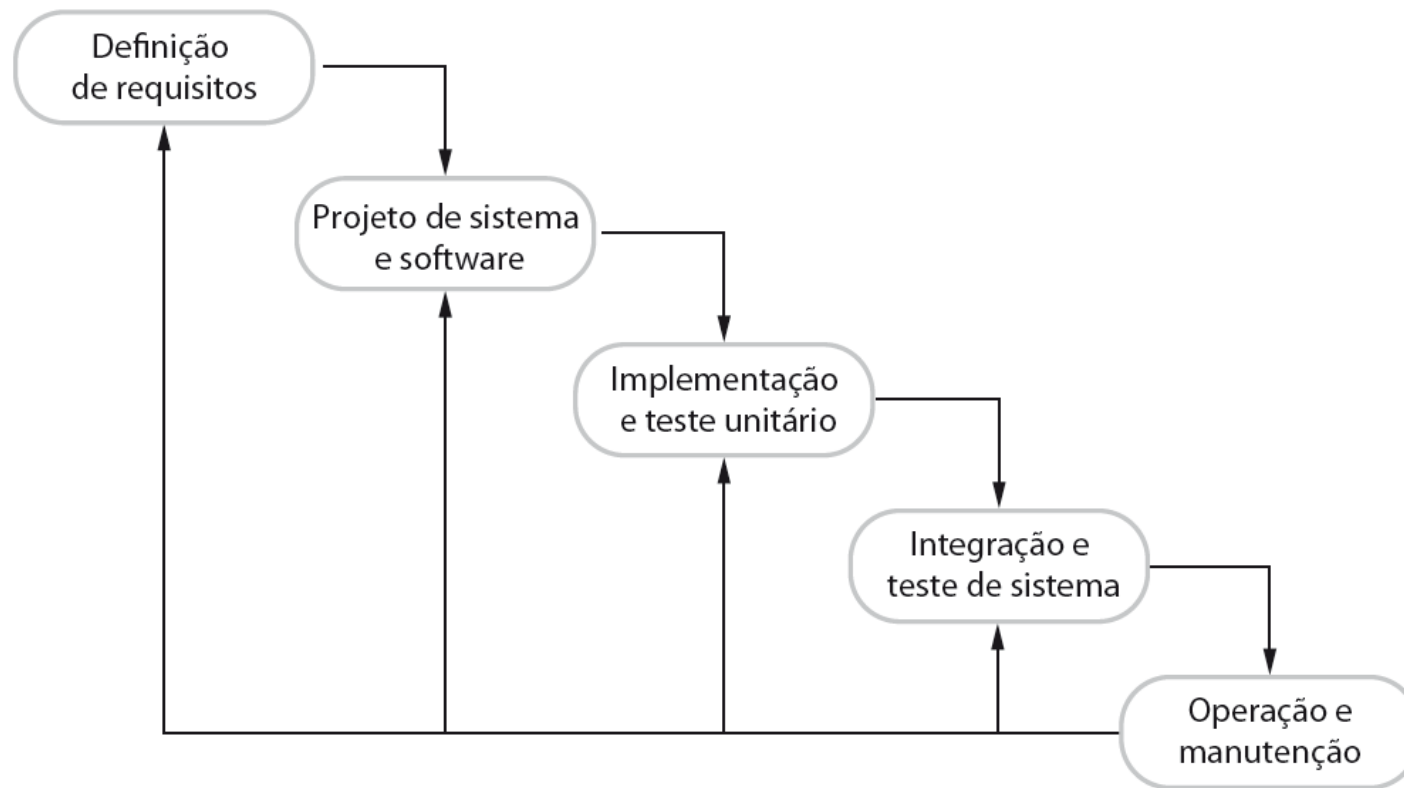
1. PROCESSOS E MODELOS DE PROCESSOS



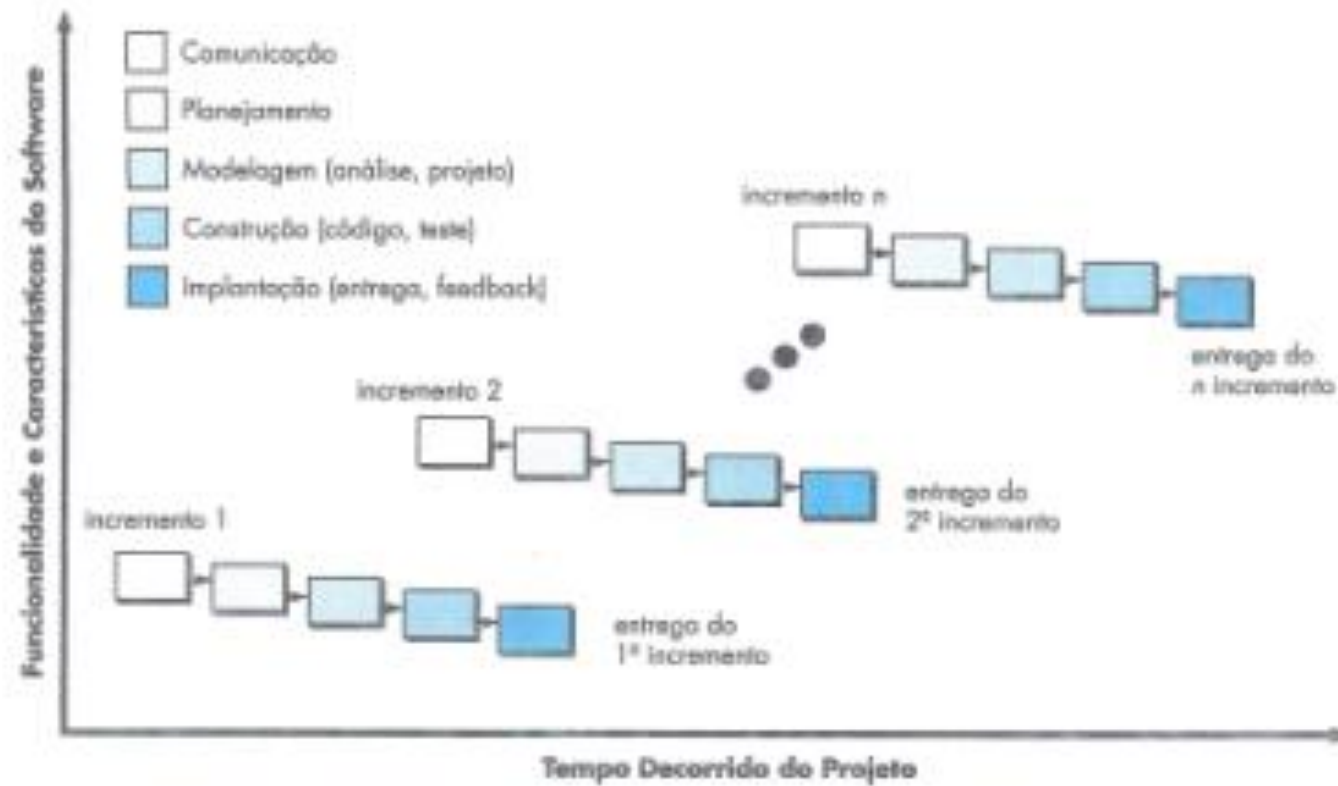
- Um modelo de processo de desenvolvimento de software é uma representação abstrata de um processo.
 - Modelo Cascata
 - Desenvolvimento Incremental



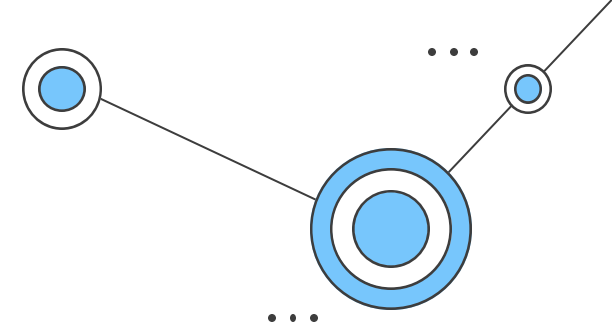
1. PROCESSOS E MODELOS DE PROCESSOS



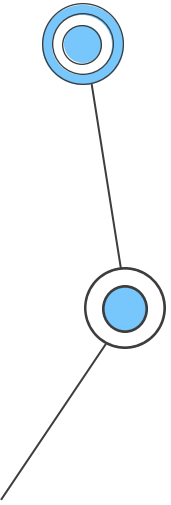
1. PROCESSOS E MODELOS DE PROCESSOS



2. METODOLOGIAS ÁGEIS



- **Processos dirigidos a planos** são processos em que todas as atividades do processo são **planejadas com antecedência** e o progresso é medido em relação a esse plano.
- Nos **processos ágeis o planejamento é incremental** e é mais fácil modificar o processo para refletir alterações nos requisitos do cliente.



2. METODOLOGIAS ÁGEIS

Manifesto para o desenvolvimento ágil de software

Estamos descobrindo maneiras melhores de desenvolver software fazendo-o nós mesmos e ajudando outros a fazê-lo. Através deste trabalho, passamos a valorizar:

Indivíduos e interação entre eles mais que processos e ferramentas
Software em funcionamento mais que documentação abrangente
Colaboração com o cliente mais que negociação de contratos
Responder a mudanças mais que seguir um plano

Mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

Kent Beck

Mike Beedle

Arie van Bennekum

Alistair Cockburn

Ward Cunningham

Martin Fowler

James Grenning

Jim Highsmith

Andrew Hunt

Ron Jeffries

Jon Kern

Brian Marick

Robert C. Martin

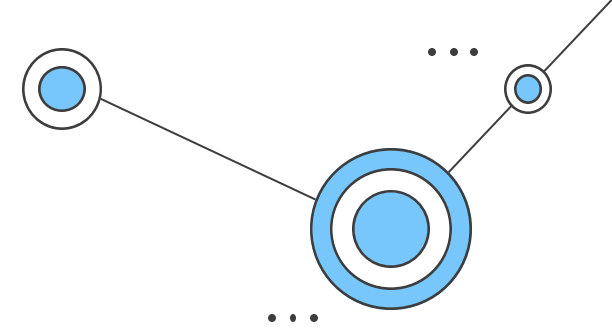
Steve Mellor

Ken Schwaber

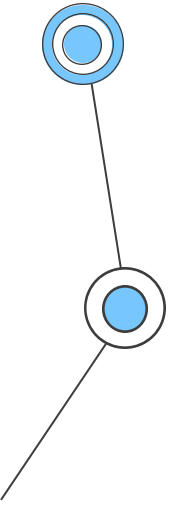
Jeff Sutherland

Dave Thomas

2. METODOLOGIAS ÁGEIS

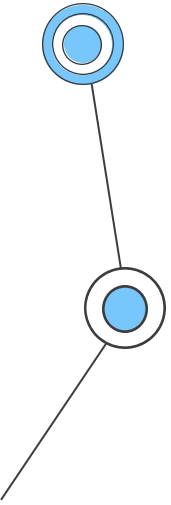
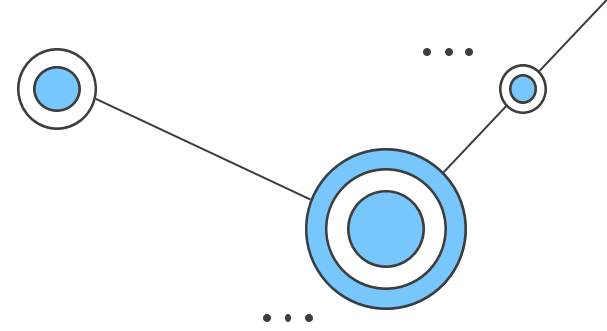


- Aplicabilidade
 - Desenvolvimento de produto pequeno ou médio;
 - Desenvolvimento de sistema personalizado dentro de uma organização (cliente envolvido);
 - Foco em equipes pequenas e fortemente integradas.



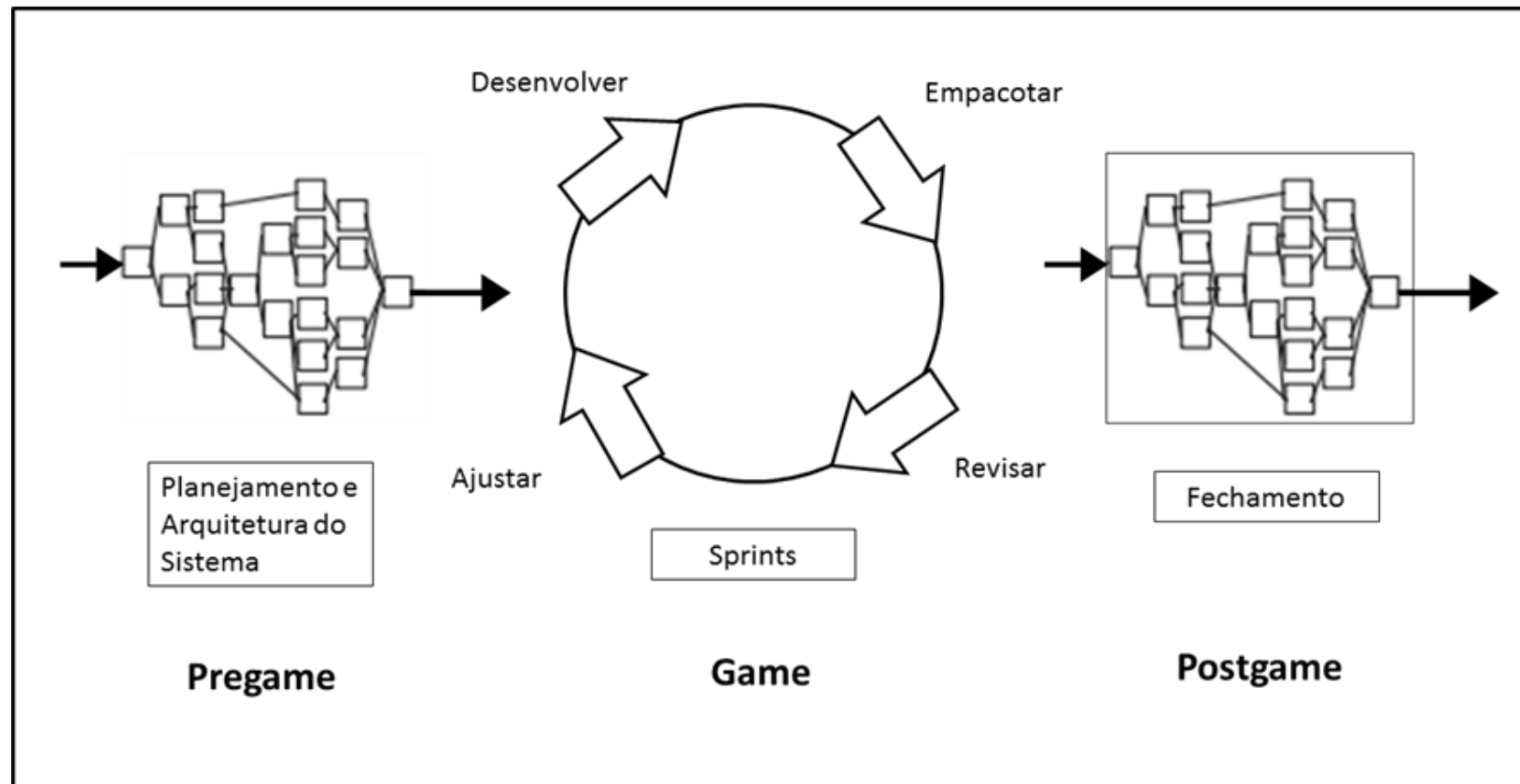
2. METODOLOGIAS ÁGEIS

- Desafios
 - Difícil manter interesse do cliente.
 - Membros da equipe não adaptados ao envolvimento intenso.
 - Manter simplicidade requer trabalho extra.
 - Os contratos podem ser um problema

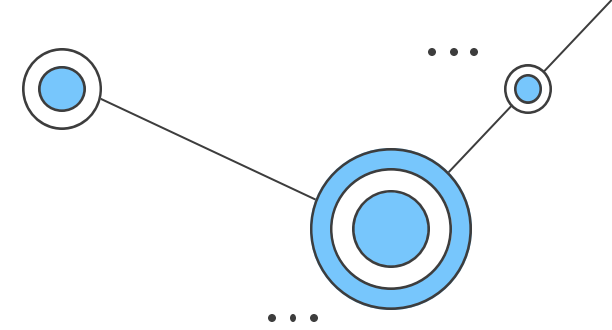


2. METODOLOGIAS ÁGEIS

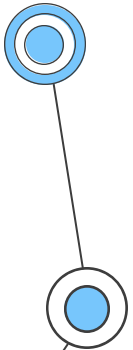
- SCRUM



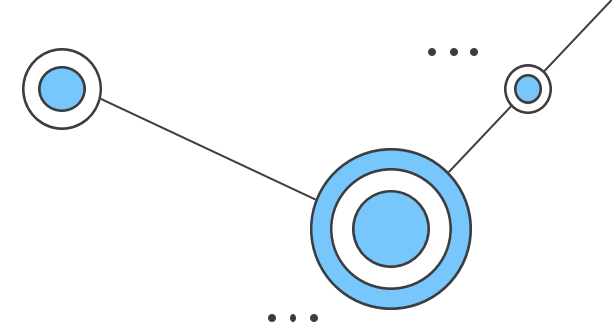
3. REQUISITOS DE SOFTWARE



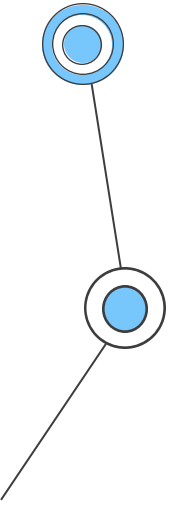
- Requisitos definem **o que um sistema deve fazer e sob quais restrições**.
 - Requisitos relacionados com a primeira parte dessa definição — o que um sistema deve fazer, ou seja, suas funcionalidades — são chamados de **Requisitos Funcionais**.
 - Já os requisitos relacionados com a segunda parte — sob que restrições — são chamados de **Requisitos Não-Funcionais**.



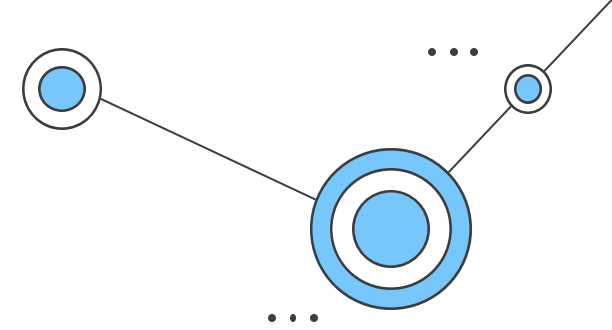
3. REQUISITOS DE SOFTWARE



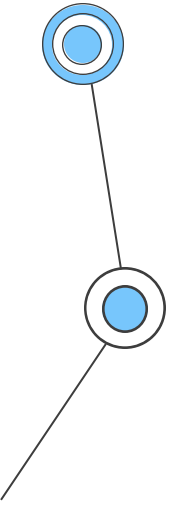
- **Engenharia de Requisitos** é o nome que se dá ao conjunto de atividades relacionadas com a descoberta, análise, especificação e manutenção dos requisitos de um sistema.
 - Elicitação e análise de requisitos;
 - Validação de requisitos;
 - Gerenciamento de requisitos.



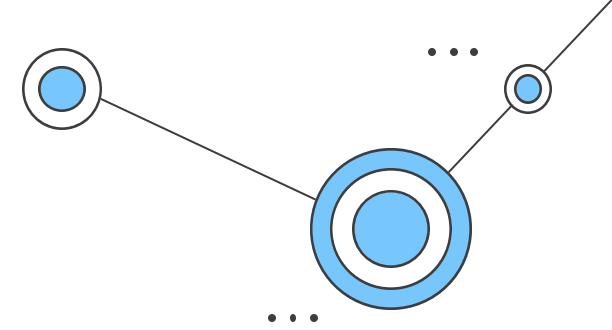
3. REQUISITOS DE SOFTWARE



- Na medida do possível, o **documento de requisitos** deve definir O QUE o sistema deve fazer ao invés de COMO deve fazê-lo.
 - Histórias de Usuário
 - Casos de Uso



3. REQUISITOS DE SOFTWARE



Como usuário típico, eu gostaria de realizar empréstimos de livros

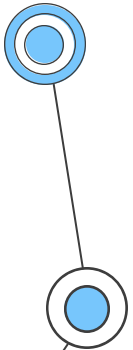
Como usuário típico, eu gostaria de devolver um livro que tomei emprestado

Como usuário típico, eu gostaria de renovar empréstimos de livros

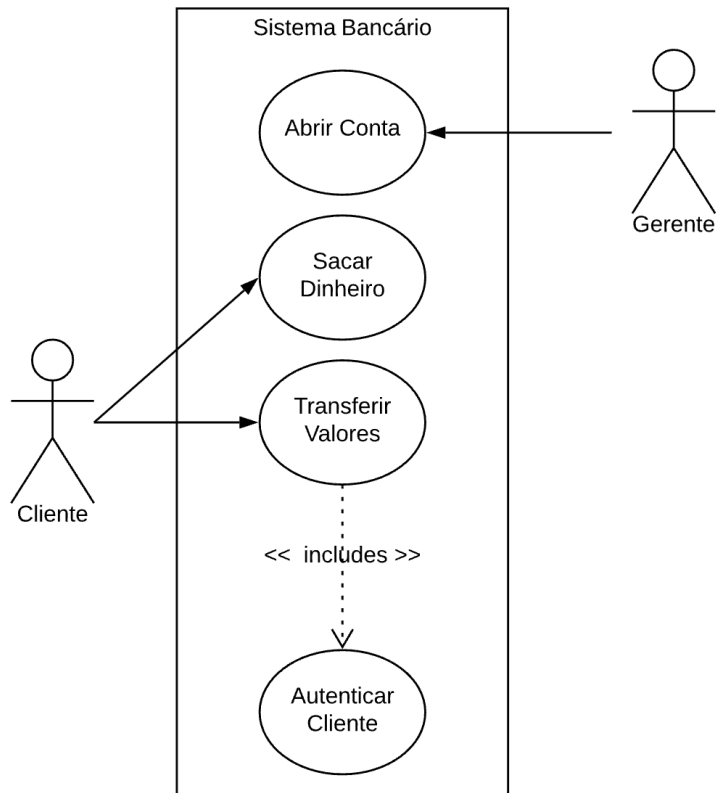
Como usuário típico, eu gostaria de pesquisar por livros

Como usuário típico, eu gostaria de reservar livros que estão emprestados

Como usuário típico, eu gostaria de receber e-mails com novas aquisições



3. REQUISITOS DE SOFTWARE



Transferir Valores entre Contas

Ator: Cliente do Banco

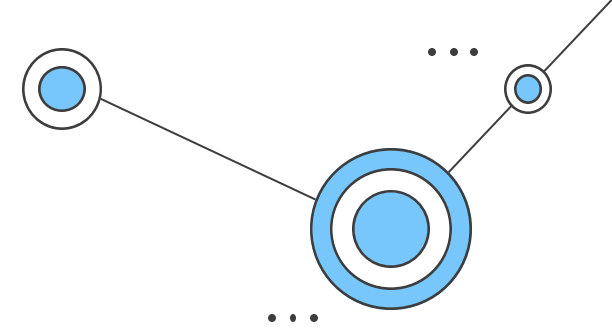
Fluxo normal:

- 1 - Autenticar Cliente (sublinhado)
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Cliente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação
- 5 - Sistema efetua transferência
- 6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

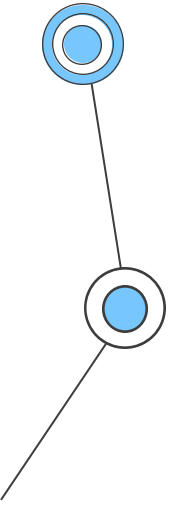
Extensões:

- 2a - Se conta e agência incorretas, solicitar nova conta e agência
- 3a - Se valor acima do saldo atual, solicitar novo valor
- 4a - Data informada deve ser a data atual ou no máximo um ano a frente
- 5a - Se data informada é a data atual, transferir imediatamente
- 5b - Se data informada é uma data futura, agendar transferência

4. PROJETO DE SOFTWARE

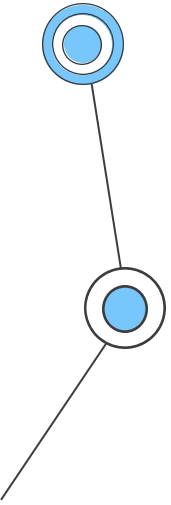
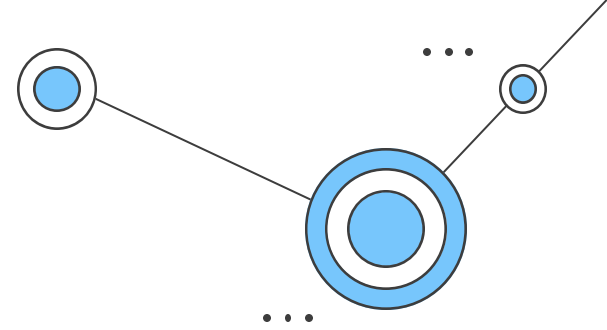


- “Existe uma diferença gritante entre construir uma pequena casa e um prédio de vários andares.”
- Sem projeto, nos arriscamos a construir um sistema instável:
 - que falhará quando pequenas modificações foram feitas;
 - que pode ser difícil de testar;
 - cuja qualidade não pode ser avaliada até certo ponto do processo de desenvolvimento.

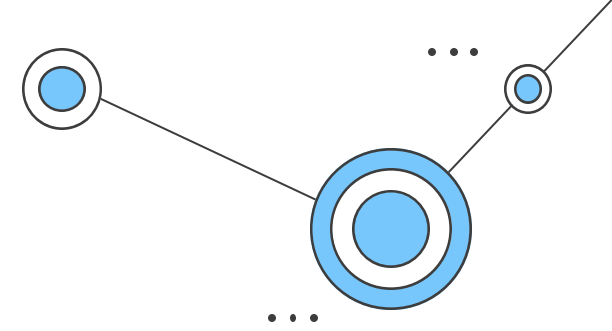


4. PROJETO DE SOFTWARE

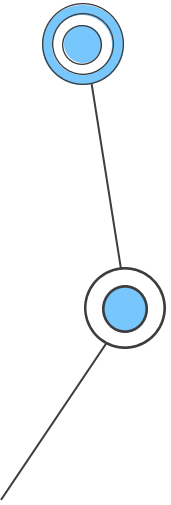
- Modelos de projetos:
 - Projeto de interface;
 - Projeto de dados;
 - Projeto em nível de componentes;
 - Projeto arquitetural.



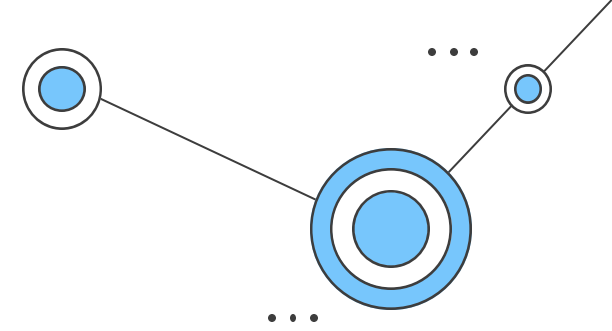
4. PROJETO DE SOFTWARE



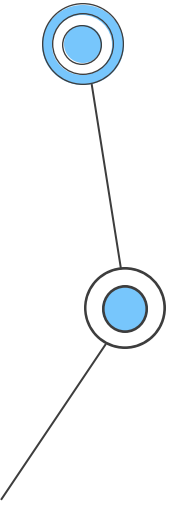
- A **arquitetura de software** de um programa ou sistema computacional é a estrutura (ou estruturas), a qual envolve os elementos de software, as propriedades desses elementos visíveis externamente e seus respectivos inter-relacionamentos.



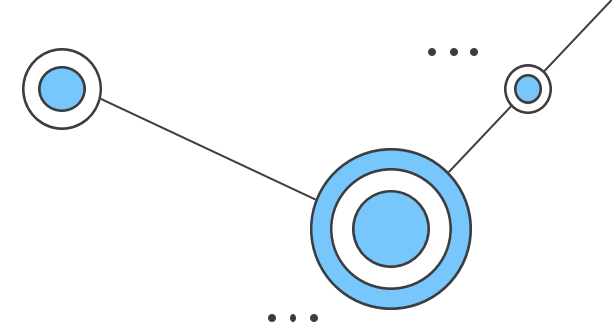
4. PROJETO DE SOFTWARE



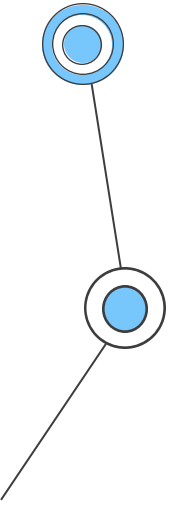
- Padrões arquiteturais propõem uma organização de mais alto nível para sistemas de software, incluindo seus principais módulos e as relações entre eles.
- Arquitetura em Camadas
- MVC
- Microserviços
- Arquiteturas orientadas à mensagens
- ...



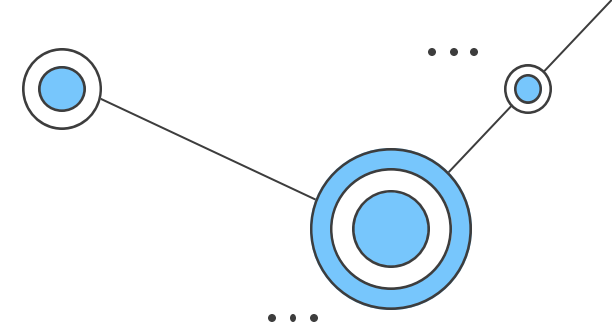
4. PROJETO DE SOFTWARE



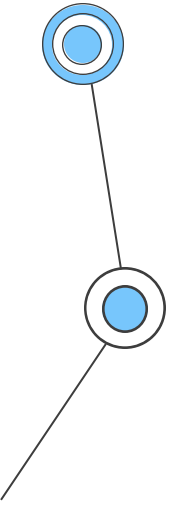
- Unified Modeling Language
 - UML não é uma linguagem de programação!
 - UML não é um processo de desenvolvimento de software!
- É uma notação gráfica utilizada para representar modelos de software.



5. QUALIDADE DE SOFTWARE



- Você estaria satisfeito com um nível de qualidade de 99,9%?
 - 20.000 prescrições médicas erradas por ano;
 - Beber água não confiável uma hora por mês;
 - Nenhum serviço telefônico durante 10 minutos por semana;
 - Falta de água e luz 10 horas por ano;
 - 500 cirurgias incorretas por semana;

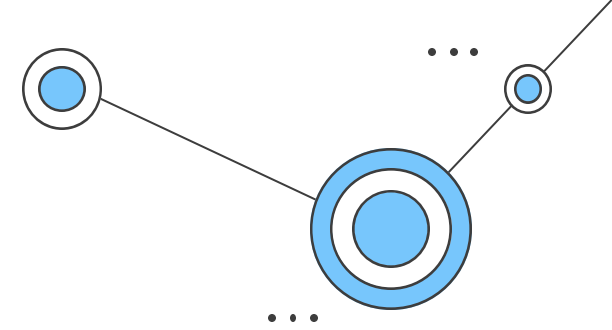


5. QUALIDADE DE SOFTWARE

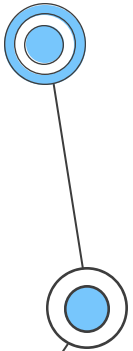
- Preocupados em garantir que o nível necessário de qualidade seja alcançado em um produto de software.



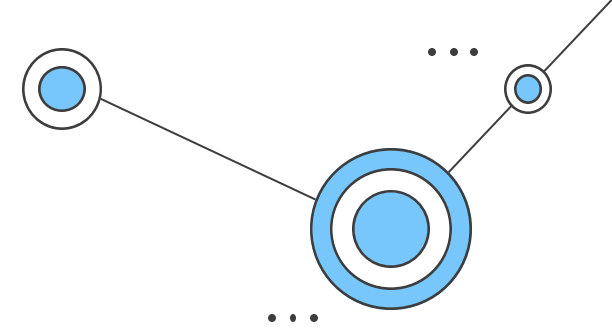
5. QUALIDADE DE SOFTWARE



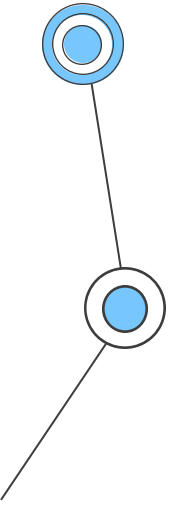
- Produto
 - ISO 9126 - Norma para qualidade de produtos de software
 - ISO 14598 - Guias para avaliação de produtos de software
 - ISO 25000 (SQuaRE) - Reformulação da ISO 9126+14598
- Processo
 - ISO 90003 - Diretrizes para aplicação da norma ISO 9001 ao desenvolvimento, fornecimento e manutenção de software.
 - ISO 12207 - Processos de ciclo de vida do software.
 - ISO 15504 (SPICE) - Projeto da ISO/IEC para avaliação dos processos de desenvolvimento de software.
 - ISO 330XX - série de normas em substituição à série ISO 15504.
 - CMMI - Capability Maturity Model Integrated. Modelo que estende o CMM para avaliação de processos de software.
 - MPS.BR - Modelo Brasileiro de qualidade de processo de software, baseado nas normas ISO 12207 e ISO 330XX, e no modelo CMMI.



5. QUALIDADE DE SOFTWARE



- Deve existir uma certificação oficial emitida com base em um padrão;
- Possíveis problemas:
 - Podem não ser percebidos como relevantes e atualizados pelos engenheiros de software.
 - Muitas vezes eles envolvem o preenchimento de formulários muito burocráticos.



5. QUALIDADE DE SOFTWARE

- CMMI

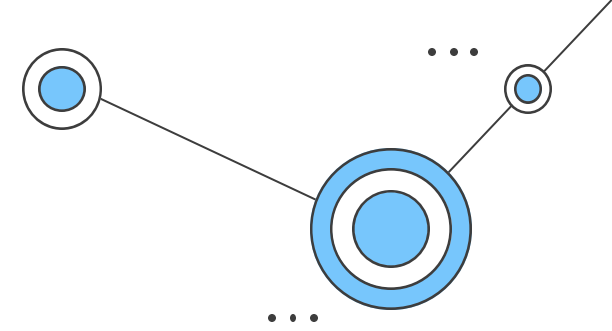


5. QUALIDADE DE SOFTWARE

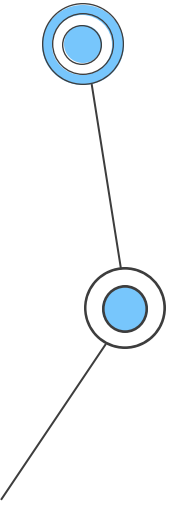
- MPS.BR



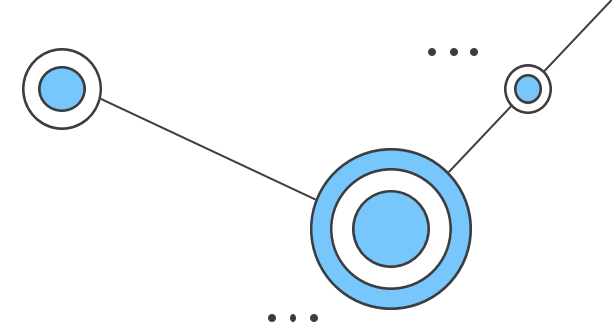
6. TESTE DE SOFTWARE



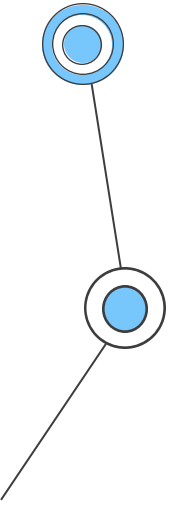
- Os testes são destinados a **mostrar o que um programa faz, o que pretende fazer** e para **descobrir os defeitos** do programa antes desse ser colocado em uso.
- Ao testar o software, você executa um programa usando **dados artificiais**.
- Podem revelar a **presença de erros, NÃO a sua ausência**.



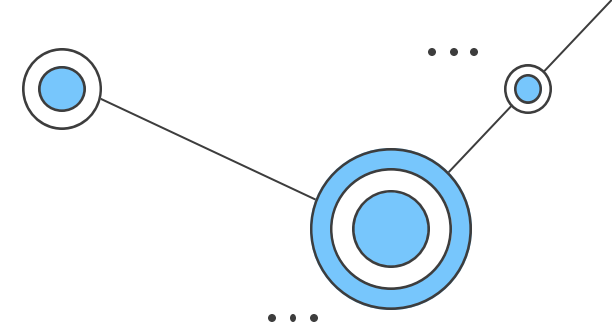
6. TESTE DE SOFTWARE



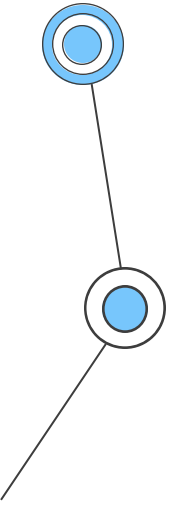
- Quando um BUG ocorre?
 - O software NÃO faz algo que a especificação diz que ele deveria fazer;
 - O software FAZ algo que a especificação diz que ele NÃO deveria fazer;
 - O software faz algo que a especificação não menciona;
 - O software NÃO faz algo que a especificação NÃO menciona, mas deveria mencionar;
 - O software é difícil de usar, entender, ou na visão do testador pode ser visto pelo usuário final como não estando correto.



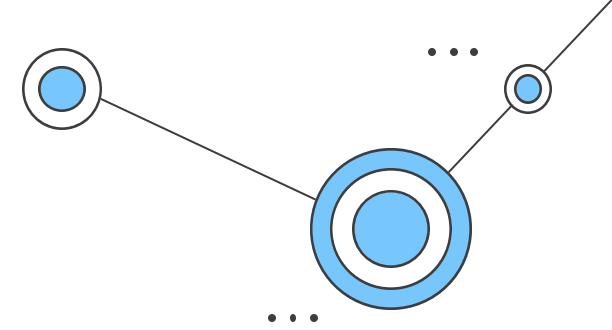
6. TESTE DE SOFTWARE



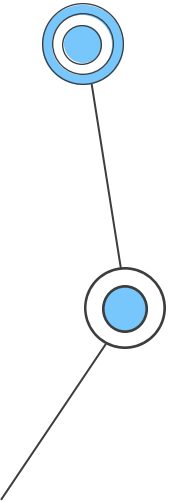
- Verificação:
 - "Estamos construindo o produto da maneira correta?".
 - O software deve estar em **acordo com sua especificação**.
- Validação:
 - "Estamos construindo o produto certo?".
 - O software deve fazer **o que o usuário realmente necessita**.



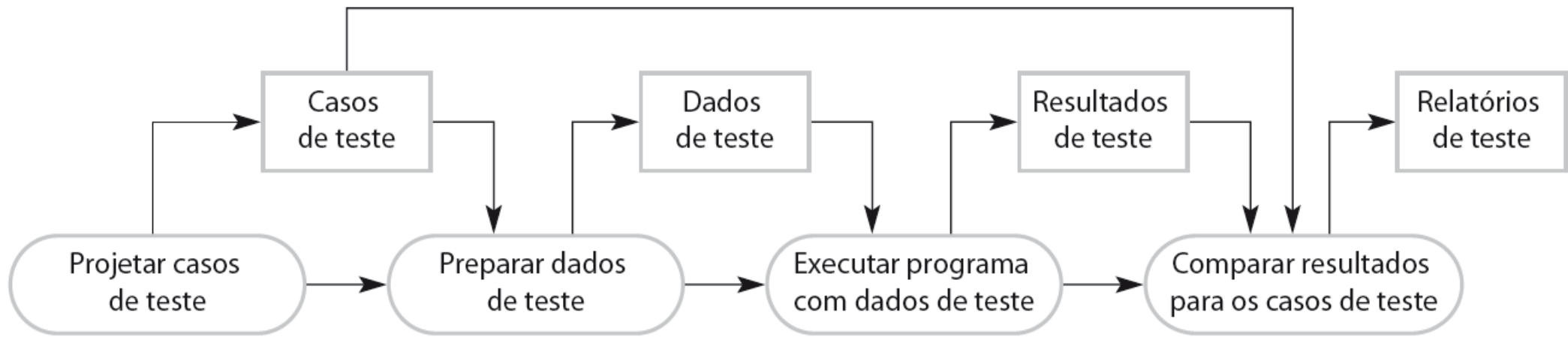
6. TESTE DE SOFTWARE



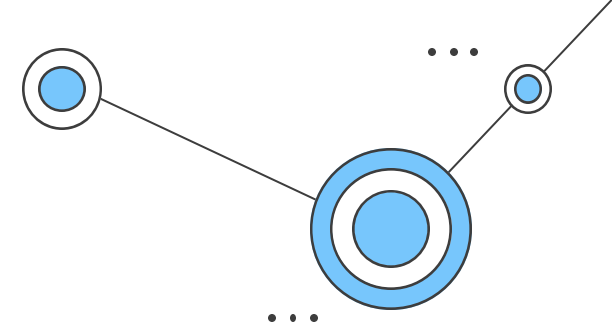
Test Case Tradicional	
ID	TCS-123
Nome	Consultar um Triângulo Escaleno
Ambiente	RVM-1.8.7
Ator	Aluno de Matemática
Pré-Condições	Não existir nenhum triângulo
Procedimentos (Entradas e Saídas)	1 – Abra a aplicação 2 – Solicite consultar o tipo de um triângulo 3 – O sistema solicita os tamanhos dos lados do triângulo 4 – Informe o lado A como "3" 5 – Informe o lado B como "4" 6 – Informe o lado C como "5" 7 – O sistema informa que o triângulo é "Escaleno"
Pós-Condições	Existe um triângulo do tipo escaleno



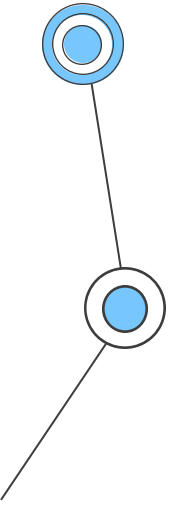
6. TESTE DE SOFTWARE



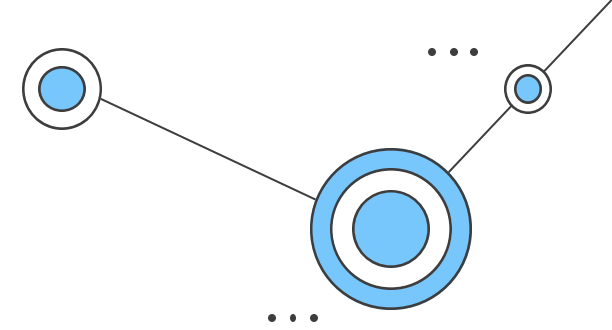
7. GERÊNCIA DE CONFIGURAÇÃO E MUDANÇA



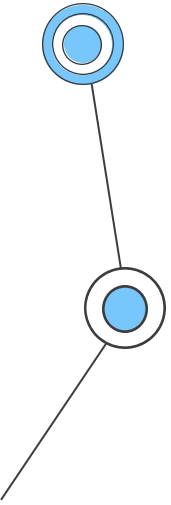
- Gerenciamento de configuração está relacionado a um conjunto de políticas, processos e ferramentas para o **gerenciamento de sistemas de software que sofrem mudanças**.
- Para que serve?
 - Software mudam constantemente;
 - Mudanças levam a um conjunto de versões;
 - É necessário gerenciar essas versões;
 - Software se desenvolve em equipes trabalhando em arquivos comuns.



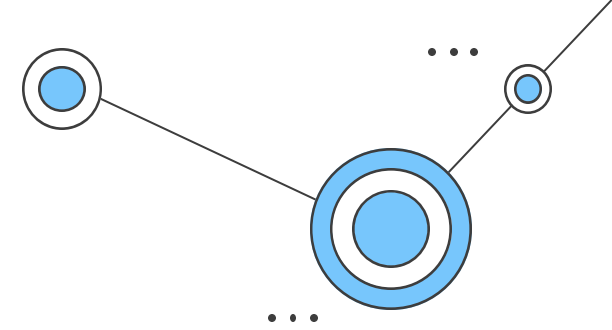
7. GERÊNCIA DE CONFIGURAÇÃO E MUDANÇA



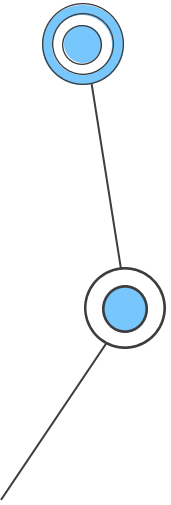
- Gerenciamento de mudanças
 - Visa garantir que a evolução do sistema seja um processo gerenciado e que seja dada prioridade às mudanças mais urgentes e de custo-benefício.
 - Acompanhar mudanças;
 - Definir custos e impacto;
 - Decidir que mudanças implementar.



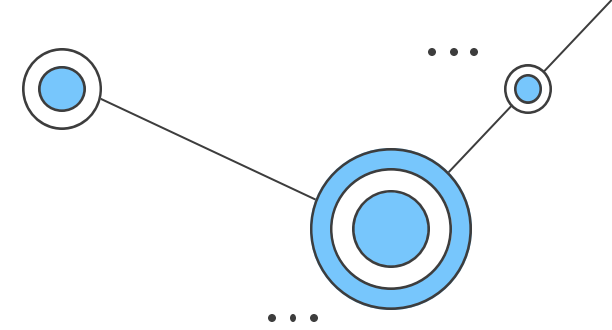
7. GERÊNCIA DE CONFIGURAÇÃO E MUDANÇA



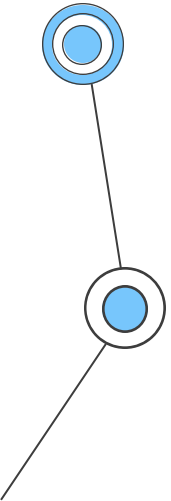
- Gerenciamento de versões
 - Controle de múltiplas versões;
 - Alterações feitas por múltiplos desenvolvedores não interfiram umas nas outras;



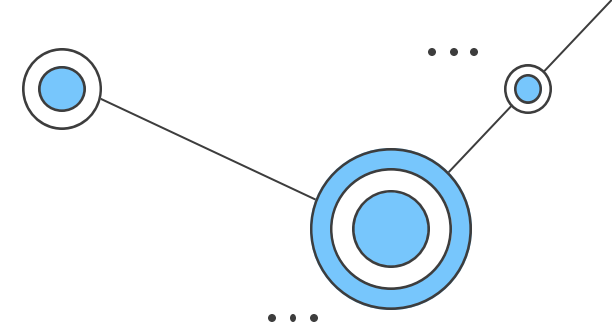
7. GERÊNCIA DE CONFIGURAÇÃO E MUDANÇA



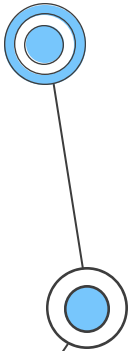
- Construção do Sistema
 - Montagem dos componentes, dados e bibliotecas;
 - Compilação do executável;
 - Geração de script de construção;
 - Automação de testes;
 - Emissão de relatórios;
 - Geração de documentação;
 - Integração Contínua.



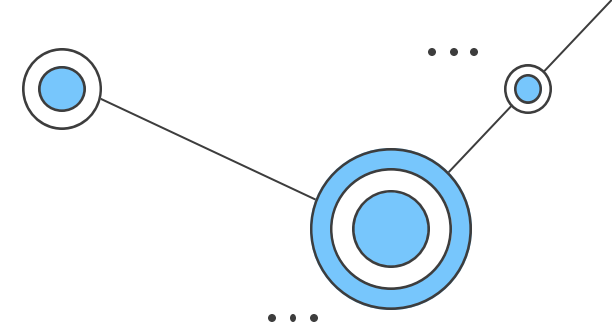
7. GERÊNCIA DE CONFIGURAÇÃO E MUDANÇA



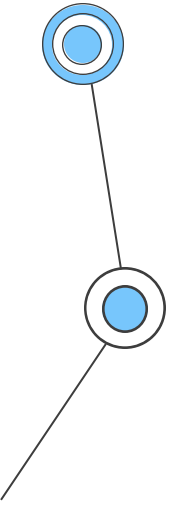
- Gerenciamento de Release
 - Um release de um sistema é uma versão de um sistema de software distribuído aos clientes.
 - Preparar o software para release;
 - Acompanhar as versões que já foram liberadas.



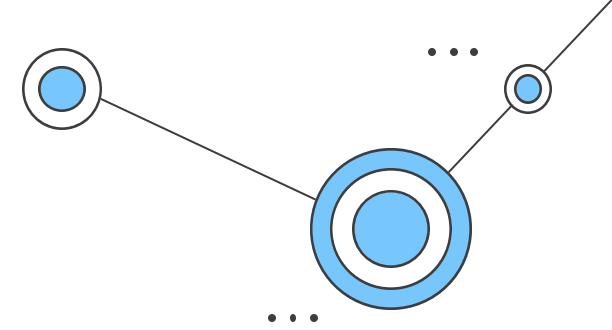
8. MANUTENÇÃO DE SOFTWARE



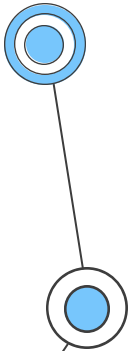
- Modificar um programa depois desse ter sido liberado para uso.
- Geralmente, a manutenção **não envolve mudanças importantes para a arquitetura do sistema.**
- As mudanças são implementadas por meio da **modificação dos componentes existentes e pela adição de novos componentes ao sistema.**



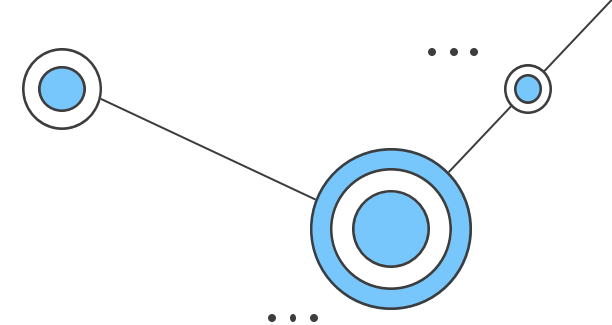
8. MANUTENÇÃO DE SOFTWARE



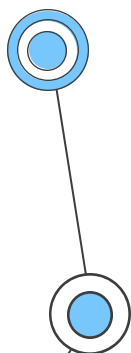
- Necessidade de manutenção...
 - Novos requisitos emergem quando o software é usado;
 - Mudanças no ambiente de negócios;
 - Erros devem ser reparados;
 - Computadores e equipamentos novos são adicionados ao sistema;
 - O desempenho ou a confiabilidade do sistema pode demandar melhorias.



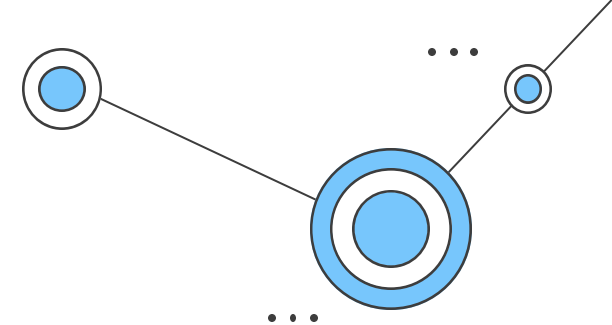
8. MANUTENÇÃO DE SOFTWARE



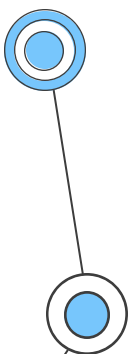
- O termo sistema legado descreve um sistema antigo que permanece em operação em uma organização.
 - Idade
 - Manutenção difícil
 - Falta de flexibilidade
 - Dependência de hardware ou software específico
 - Interface de usuário desatualizada
 - Falta de integração com outros sistemas
 - Altos custos de manutenção

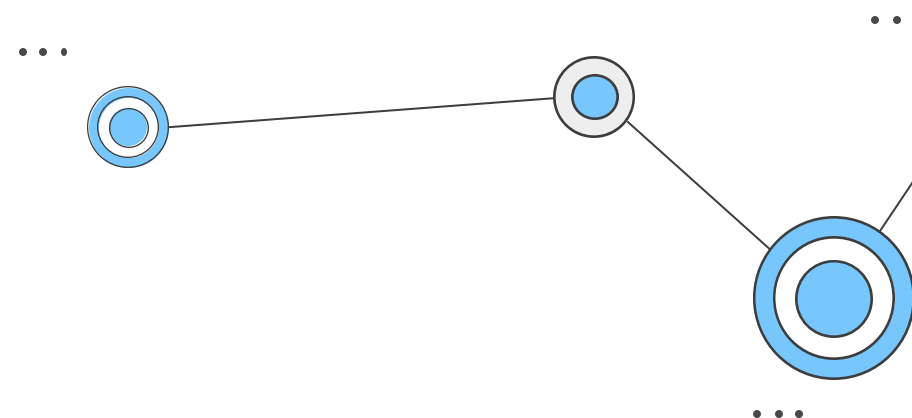


8. MANUTENÇÃO DE SOFTWARE



- **Reengenharia:** reestruturação ou reescrita de parte ou da totalidade de um sistema legado, sem alterar sua funcionalidade.
- **Refatoração:** processo de fazer melhorias em um programa para diminuir a degradação por meio da mudança.
- **Code Smell (cheiro de código):** qualquer característica no código-fonte de um programa que possivelmente indica um problema mais profundo.





Conceitos Básicos da ES

